

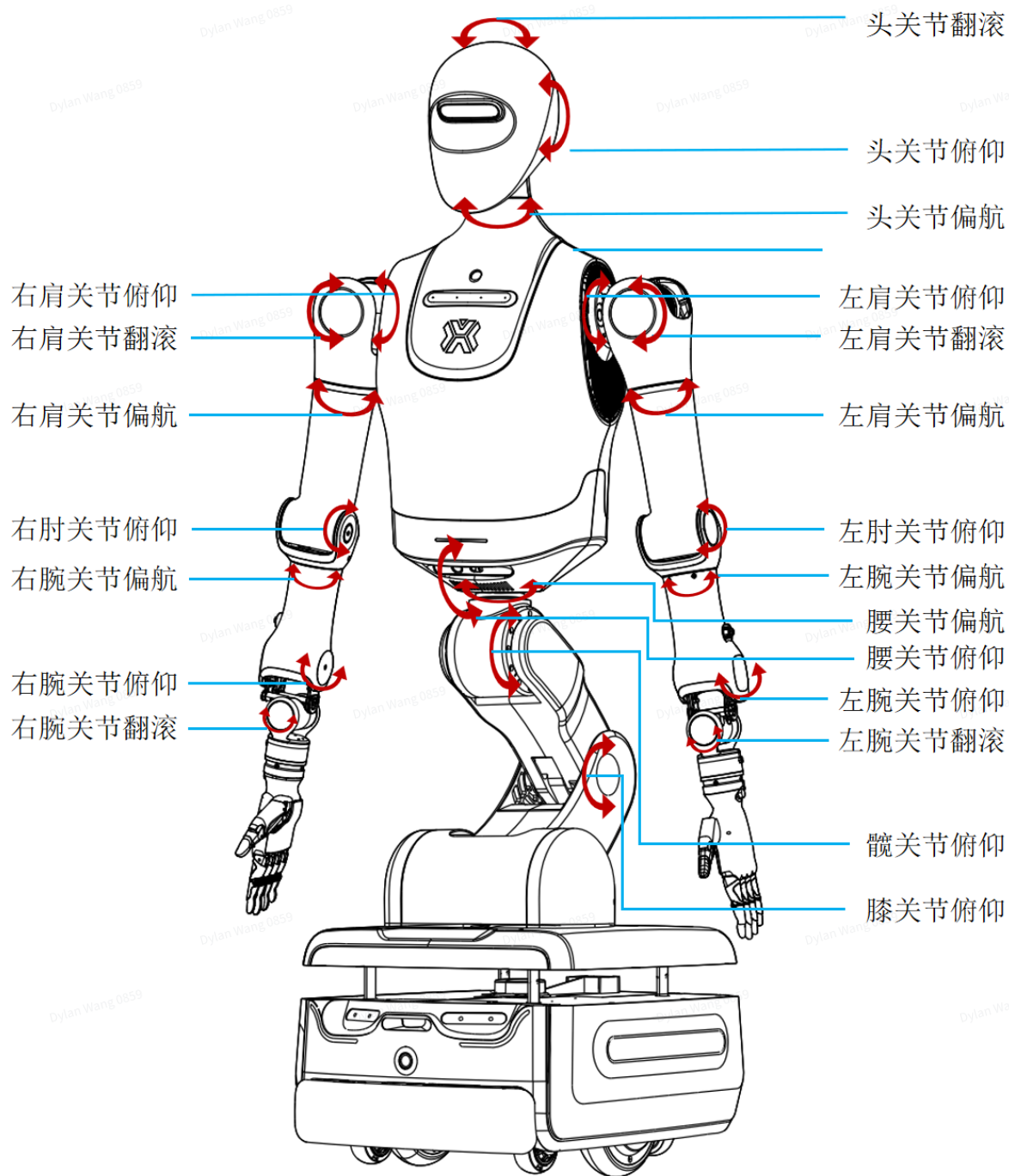
Pro版天铁2.0 ROS2 SDK二次开发文档

适用软件版本	v2.0.5
文档更新时间	2025-09-24

1. 关于天铁2.0 Pro版

Pro版天铁2.0 整机分为上半身、下半身和一个底盘，具备多个自由度。头部拥有3个自由度，单手臂拥有7个自由度，包括肩关节、肘关节和腕关节。腰部有2个自由度。腿部拥有2个自由度，包括髋关节和膝关节。底盘有2个自由度，可360度自由旋转。整机共有35个自由度，使得机器人能够实现精确的运动和姿态控制。

1.1 全身关节参数



所属部位	名称	运动范围 (°)	扭矩系数 (Nm/A)	最大转速 (rpm)	KP/KD范围	额定力矩 (Nm)	峰值力矩 (Nm)	关节名称
头部	头关节翻滚 Head Roll	-26°~+26°	2.16	64	Kp: 0~2000 kd: 0~300	3	9	1
	头关节俯仰 Head Pitch	-25°~+25°	2.16	64	Kp: 0~2000	3	9	2

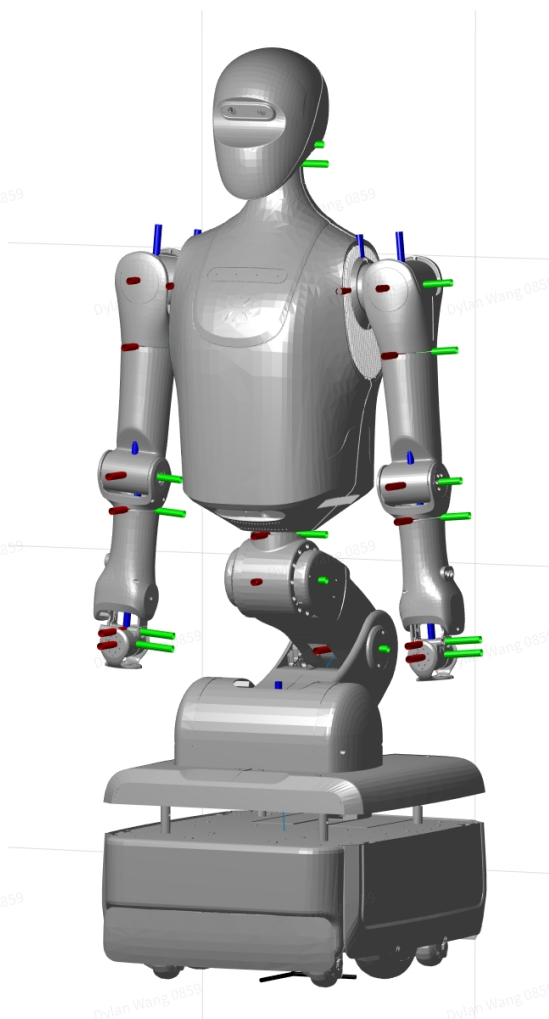
					kd: 0~300			
	头关节偏航 Head Yaw	-90°~+90°	2.16	64	Kp: 0~2000 kd: 0~300	3	9	3
腰部	腰关节偏航 Waist Yaw	-160°~+180°	/	30	/	31	143	31
	腰关节俯仰 Waist Pitch	-45°~+120°	/	37.5	/	82	332	32
左臂	左肩关节俯仰 Left Shoulder Pitch	-170°~+170°	3.207	88	Kp: 0~2000 kd: 0~300	35	91	11
	左肩关节翻滚 Left Shoulder Roll	-15°~+150°	3.37	120	Kp: 0~2000 kd: 0~300	20	60	12
	左肩关节偏航 Left Shoulder Yaw	-170°~+170°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	13
	左肘关节俯仰 Left Elbow Pitch	-150°~+15°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	14
	左腕关节偏航 Left Wrist Yaw	-170°~+170°	1.71	146	Kp: 0~2000 kd: 0~300	5	15	15
	左腕关节俯仰 Left Wrist Pitch	-45°~+60°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	16
	左腕关节翻滚 Left Wrist Roll	-95°~+75°	2.16	72	Kp: 0~2000	3	9	17

					kd: 0~300			
右臂	右肩关节俯仰 Right Shoulder Pitch	-170°~+170°	3.207	88	Kp: 0~2000 kd: 0~300	35	91	21
	右肩关节翻滚 Right Shoulder Roll	-150°~+15°	3.37	120	Kp: 0~2000 kd: 0~300	20	60	22
	右肩关节偏航 Right Shoulder Yaw	-170°~+170°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	23
	右肘关节俯仰 Right Elbow Pitch	-150°~+15°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	24
	右腕关节偏航 Right Wrist Yaw	-170°~+170°	1.71	146	Kp: 0~2000 kd: 0~300	5	15	25
	右腕关节俯仰 Right Wrist Pitch	-45°~+60°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	26
	右腕关节翻滚 Right Wrist Roll	-75°~+95°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	27
腿部	髋关节俯仰 Hip Pitch	+13°~+80°	/	37.5	/	332	332	51
	膝关节俯仰 Knee Pitch	+26°~+160°	/	37.5	/	332	332	52

1.2 整机urdf，坐标系，关节旋转轴与关节零点

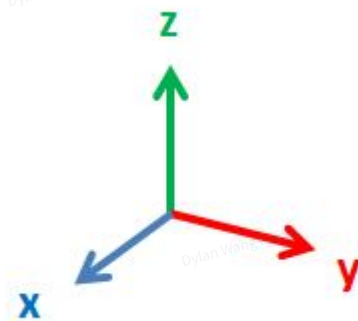
1.2.1 关节零点

如下图所示：

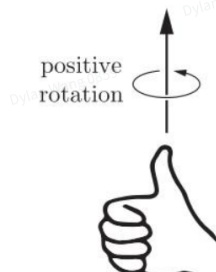
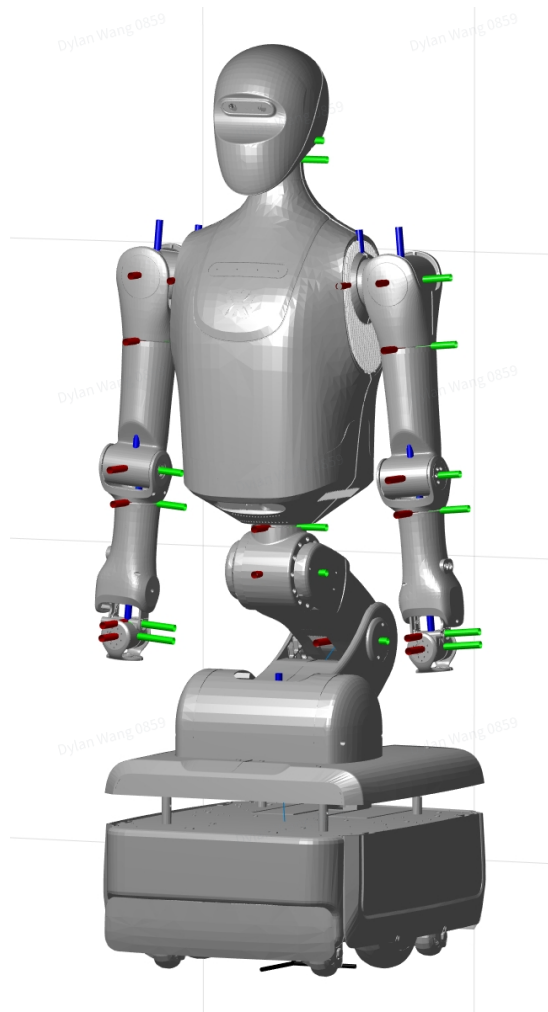


1.2.2 坐标系

- a. 整机坐标系-X轴（Roll）以机器人前方为正方向；
- b. 整机坐标系-Y轴（Pitch）以机器人左方为正方向；
- c. 整机坐标系-Z轴（Yaw）以机器人上方为正方向；



1.2.3 旋转轴 & 正方向



2. 快速操作指南

⚠注意：机器人目前有两种启动方式：自启动和手动程序启动。

如启用自启动，则✅需执行步骤 2.d. 按A键自检，❌无需执行步骤 4. 程序启动；

如关闭自启动，手动程序启动，则❌无需执行步骤 2.d. 按A键自检，✅需执行步骤 4. 程序启动。

1. 将遥控器上的电源键先短按后立即长按，打开遥控器；

2. 开机上电：

- 按下总开关键；
- 短按开机按钮；
- 顺时针旋转急停按钮直至弹出；
- （自启动启用的前提下）** 等待机器人开机完成后（即开机音效结束），在 F 拨杆置于中间挡位的前提下，按遥控器 A 键启动自检，自检成功后（即整机状态灯变为蓝绿常亮状态）进入可操作的Ready状态。

3. 主控板登录和连接

• 配置Wi-Fi步骤：

- 用以太网线连接机器人背后的调试以太网接口和用户电脑端；

- b. 用户电脑配置以太网端口地址为192.168.41.xx/255.255.255.0;
- c. 打开终端中输入 `ssh ubuntu@192.168.41.x` 登录需要配置的主控板（运控x86和orin，其IP分别为192.168.41.1和192.168.41.2）；
- d. 在上一步的终端中继续依次输入如下命令配置IP：

代码块

```
1 sudo nmcli device wifi list #查看Wi-Fi热点的SSID
2 sudo nmcli device wifi connect 'Wi-Fi SSID' password 'Wi-Fi密码' #连接Wi-Fi
3 ifconfig #检查是否连接成功
```

e. Wi-Fi配置完成，拔掉网线。

- **Wi-Fi配置完成的前提下，每次机器人无需再配置Wi-Fi，直接用Wi-Fi连接的步骤如下：**

- a. 输入 `ifconfig`，查看给主控板配置的IP地址；
- b. 输入 `ssh ubuntu@x.x.x.x` 登录连接主控板服务器，其中 `x.x.x.x` 为上一步获取的IP地址。随后可进行下一步程序启动。

4. （自启动关闭的前提下）程序启动

- a. 手动启动请先确保已经关闭自启动功能：

代码块

```
1 sudo systemctl stop proc_manager.service
```

- b. 首先，打开第一个终端，依次输入如下命令启动**主控节点**：

代码块

```
1 sudo su
2 cd ros2ws
3 source install/setup.bash
4 ros2 launch body_control body.launch.py
```

此时可以开始使用 SDK 进行开发调试。

5. 关机：

- a. 确认机器人已停止运动；
- b. 按下急停按钮；
- c. 按下主控开关键；

- d. 按下电池开关键；
- e. 将遥控器上的电源键先短按后长按，关闭遥控器。

3. 全局值说明

3.1 全身关节id定义

- 消息名称：MotorName
- 数据定义位置：MotorName.msg
- 说明：Pro版天铁头部、臂部、腰部和腿部的电机的命名ID定义如下。

代码块

```
1  # head 00 + id
2  uint16 MOTOR_HEAD_1 = 1
3  uint16 MOTOR_HEAD_2 = 2
4  uint16 MOTOR_HEAD_3 = 3
5
6  # arm-left 10 + id
7  uint16 MOTOR_ARM_LEFT_1 = 11
8  uint16 MOTOR_ARM_LEFT_2 = 12
9  uint16 MOTOR_ARM_LEFT_3 = 13
10 uint16 MOTOR_ARM_LEFT_4 = 14
11 uint16 MOTOR_ARM_LEFT_5 = 15
12 uint16 MOTOR_ARM_LEFT_6 = 16
13 uint16 MOTOR_ARM_LEFT_7 = 17
14
15 # arm-right 20 + id
16 uint16 MOTOR_ARM_RIGHT_1 = 21
17 uint16 MOTOR_ARM_RIGHT_2 = 22
18 uint16 MOTOR_ARM_RIGHT_3 = 23
19 uint16 MOTOR_ARM_RIGHT_4 = 24
20 uint16 MOTOR_ARM_RIGHT_5 = 25
21 uint16 MOTOR_ARM_RIGHT_6 = 26
22 uint16 MOTOR_ARM_RIGHT_7 = 27
23
24 # body (including waist) 30 + id
25 uint16 MOTOR_WAIST_01 = 31
26 uint16 MOTOR_WAIST_02 = 32
27
28 # leg
29 uint16 MOTOR_LEG_LEFT_1 = 51
30 uint16 MOTOR_LEG_LEFT_2 = 52
```

4. 接口说明

4.1 一键启动

4.1.1 功能启用/关闭

自启动功能默认开启，使用以下方法开启、关闭：

1. 使能自启动

代码块

```
1 sudo systemctl enable proc_manager.service
```

2. 禁用自启动

代码块

```
1 sudo systemctl disable proc_manager.service
```

4.2 关节温度获取接口

- 说明：所有关节温度获取，以 topic 形式发布，每秒 1 次；
- 数据格式：

代码块

```
1 # bodyctrl_msgs/msg/MotorStatusMsg1
2 std_msgs/Header header
3 MotorStatus1[] status
4
5 # bodyctrl_msgs/msg/MotorStatus1
6 uint16 name # MotorName
7 float32 motortemperature
8 float32 mostemperature
```

1. 获取头部关节温度信息

话题名称： `/head/motor_status`

2. 获取腰部关节温度信息

话题名称: `/waist/motor_status`

3. 获取双臂关节温度信息

话题名称: `/arm/motor_status`

4. 获取腿部关节温度信息

话题名称: `/leg/motor_status`

4.3 头部关节

4.3.1 状态获取接口

1. 获取头部关节信息

- 说明: 获取头部关节的状态信息, 其中包含关节的当前位置、速度、电流和温度。
- 控制方式: topic
- 话题名称: `/head/status`
- 数据格式:

代码块

```
1  # bodyctrl_msgs/msg/MotorStatusMsg.msg
2  std_msgs/Header header
3  MotorStatus[] status
4
5  # bodyctrl_msgs/msg/MotorStatus.msg
6  uint16 name          # MotorName
7  float32 pos          # rad
8  float32 speed        # rad
9  float32 current      # A
10 float32 temperature  # Mos温度
11 uint32 error
```

error错误字段值定义如下:

代码块

1	error = 33072,	设备掉线
2	error = 33073,	关节位置超限
3	error = 1,	关节电机过温
4	error = 2,	过流
5	error = 3,	电压过低
6	error = 4,	关节mos过温
7	error = 5,	堵转
8	error = 6,	电压过高

```
9 error = 7,      缺相
10 error = 8,     编码器错误
```

4.3.2 控制接口

1. 位置模式

- 说明：头部关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/head/cmd_pos`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 SetMotorPosition[] cmds
3
4 # SetMotorPosition.msg
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 spd # rad/s
8 float32 cur # A
```

2. 力位混合模式

- 说明：头部关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称： `/head/cmd_ctrl`
- 数据定义位置： `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 MotorCtrl[] cmds
3
4 # MotorCtrl.msg
5 uint16 name
6 float32 kp
7 float32 kd
8 float32 pos
```

```
9 float32 spd
10 float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/head/cmd_vel`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 SetMotorSpeed[] cmds
3
4 #SetMotorSpeed.msg
5 uint16 name # MotorName
6 float32 spd # rad/s
7 float32 cur # A
```

4. 关节标零

- 说明：头部关节的标零接口。
- 控制方式：topic
- 话题名称： `/head/cmd_set_zero`
- 数据定义位置： `std_msgs::msg::String`
- 数据格式：

代码块

```
1 以字符串形式发送关节的id实现标零；
2 头部关节id: 1,2,3;
```

4.4 腰部关节

4.4.1 状态获取接口

1. 获取腰部关节信息

- 说明：获取腰部关节的状态信息，其中包含关节的当前位置、速度、电流和温度。

- 控制方式: topic
- 话题名称: `/waist/status`
- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 MotorStatus[] status
3
4 MotorStatus定义如下:
5 uint16 name      # MotorName
6 float32 pos      # rad
7 float32 speed    # rad
8 float32 current  # A
9 float32 temperature #
10 uint32 error
```

error错误字段值定义如下:

代码块

```
1 error = 33072, 设备掉线
2 error = 33073, 关节位置超限
3 error = 1,     关节电机过温
4 error = 2,     过流
5 error = 3,     电压过低
6 error = 4,     关节mos过温
7 error = 5,     堵转
8 error = 6,     电压过高
9 error = 7,     缺相
10 error = 8,    编码器错误
```

4.4.2 控制接口

1. 位置模式

- 说明: 关节的位置控制接口, 需要提供期望位置、期望速度、最大电流。
- 控制方式: topic
- 话题名称: `/waist/cmd_pos`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 SetMotorPosition[] cmds
3
4 # SetMotorPosition.msg
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 spd # rad/s
8 float32 cur # A
```

2. 腰部关节标零

- 说明：腰部关节的标零接口。
- 控制方式：topic
- 话题名称： `/waist/cmd_set_zero`
- 数据定义位置： `std_msgs::msg::String`
- 数据格式：

代码块

```
1 以字符串形式发送关节的id实现标零；
2 腰部关节id: 31
```

4.5 双臂关节

4.5.1 状态获取接口

1. 获取双臂关节信息

- 说明：获取双臂关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称： `/arm/status`
- 数据定义位置： `bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 MotorStatus[] status
3
4 MotorStatus定义如下：
```

```

5  uint16 name      # MotorName
6  float32 pos      # rad
7  float32 speed    # rad
8  float32 current  # A
9  float32 temperature
10 uint32 error

```

error错误字段值定义如下：

代码块

```

1  error = 33072,    设备掉线
2  error = 33073,    关节位置超限
3  error = 1,        关节电机过温
4  error = 2,        过流
5  error = 3,        电压过低
6  error = 4,        关节mos过温
7  error = 5,        堵转
8  error = 6,        电压过高
9  error = 7,        缺相
10 error = 8,        编码器错误

```

4.5.2 控制接口

1. 位置模式

- 说明：关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/arm/cmd_pos`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```

1  std_msgs/Header header
2  SetMotorPosition[] cmds
3
4  # SetMotorPosition.msg
5  uint16 name # MotorName
6  float32 pos # rad
7  float32 spd # rad/s
8  float32 cur # A

```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称： `/arm/cmd_ctrl`
- 数据定义位置： `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  MotorCtrl[] cmds
3
4  # MotorCtrl.msg
5  uint16 name
6  float32 kp
7  float32 kd
8  float32 pos
9  float32 spd
10 float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/arm/cmd_vel`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  SetMotorSpeed[] cmds
3
4  #SetMotorSpeed.msg
5  uint16 name # MotorName
6  float32 spd # rad/s
7  float32 cur # A
```

4. 电流模式

- 说明：关节的电流控制接口，需要提供期望电流。

- 控制方式: topic
- 话题名称: `/arm/cmd_current`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorCurTor.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 SetMotorCurTor[] cmds
3
4 #SetMotorCurTor.msg
5 uint16 name # MotorName
6 float32 cur_tor
7 uint8 ctrl_status    #当前只支持电流模式
```

5. 手臂关节标零

- 说明: 双臂关节的标零接口。
- 控制方式: topic
- 话题名称: `/arm/cmd_set_zero`
- 数据定义位置: `std_msgs::msg::String`
- 数据格式:

代码块

```
1 以字符串形式发送关节的id实现标零;
2 双臂关节id:
3     左臂: 11---17,
4     右臂: 21---27;
```

4.6 腿部关节

4.6.1 状态获取接口

1. 获取双腿关节信息

- 说明: 获取双腿关节的状态信息, 其中包含关节的当前位置、速度、电流、温度和告警。
- 控制方式: topic
- 话题名称: `/leg/status`
- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg.msg`

- 数据格式：

代码块

```
1  std_msgs/Header header
2  MotorStatus[] status
3
4  MotorStatus定义如下:
5  uint16 name      # MotorName
6  float32 pos      # rad
7  float32 speed    # rad
8  float32 current  # A
9  float32 temperature
10 uint32 error
```

error错误字段值定义如下：

代码块

1	error = 33072,	设备掉线
2	error = 33073,	关节位置超限
3	error = 1,	关节电机过温
4	error = 2,	过流
5	error = 3,	电压过低
6	error = 4,	关节mos过温
7	error = 5,	堵转
8	error = 6,	电压过高
9	error = 7,	缺相
10	error = 8,	编码器错误

4.6.2 控制接口

1. 位置模式

- 说明：关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/leg/cmd_pos`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  SetMotorPosition[] cmds
```

```

3
4 # SetMotorPosition.msg
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 spd # rad/s
8 float32 cur # A

```

4.7 手部关节

4.7.1 全局值说明

- 消息名称: `MotorName`
- 数据定义位置:
- 说明: 手指各关节ID定义如下:

代码块

```

1 uint16 MOTOR_FINGER_LITTLE = 1, //little finger 小指
2 uint16 MOTOR_FINGER_RING = 2, //ring finger 无名指
3 uint16 MOTOR_FINGER_MIDDLE = 3, //middle finger 中指
4 uint16 MOTOR_FINGER_FORE = 4, //Fore finger 食指
5 uint16 MOTOR_FINGER_THUMB_BEND = 5, //thumb bend 拇指弯曲
6 uint16 MOTOR_FINGER_THUMB_ROTATION = 6, //thumb rotation 拇指旋转

```

4.7.2 状态获取接口

- 说明: 获取手指各关节的状态信息, 其中包含电机的当前位置、速度、电流。
- 控制方式: topic
- 话题名称: `/inspire_hand/state/left_hand`
- 和 `/inspire_hand/state/right_hand`
- 数据定义位置: `sensor_msgs::msg::JointState`
- 数据格式:

⚠ 注意: 这里的数据 (位置, 速度, 电流), 指的是百分比

代码块

```

1 std_msgs/Header header
2 string[] name #说明, id=1为小指, id=6为大拇指旋转
3 float64[] position #百分比
4 float64[] velocity #百分比
5 float64[] effort #百分比

```

4.7.3 控制接口-topic

1. 控制手指各关节角度

- 说明：手部电机的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/inspire_hand/ctrl/left_hand` 和 `/inspire_hand/ctrl/right_hand`
- 数据定义位置： `sensor_msgs::msg::JointState.msg`
- 数据格式：

⚠ 注意：这里的position，指的仍然是手指张开的百分比；

代码块

```
1 std_msgs/Header header
2 string[] name
3 float64[] position    #百分比
```

4.7.4 控制接口-service

1. 控制手指各关节角度

- 说明：手指各个关节的位置控制接口，以百分比的方式。
- 控制方式：service
- 服务名称： `/inspire_hand/set_angle_flexible/left_hand` 和 `/inspire_hand/set_angle_flexible/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_angle_flexible.srv`
- 数据格式：

代码块

```
1 string[] name
2 float32[] angle_ratio
3 bool angle_accepted
```

4.7.5 控制手指各关节力矩

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称： `/inspire_hand/set_force/left_hand` 和 `/inspire_hand/set_force/right_hand`

- 数据定义位置: `bodyctrl_msgs::srv::set_force.srv`

- request:

代码块

```
1 float32 force0_Ratio #小拇指-百分比 (0代表0g, 1代表1000g)
2 float32 force1_Ratio #无名指-百分比
3 float32 force2_Ratio #中指-百分比
4 float32 force3_Ratio #食指-百分比
5 float32 force4_Ratio #大拇指弯曲-百分比
6 float32 force5_Ratio #大拇指旋转-百分比
```

- response:

代码块

```
1 bool force_accepted #是否设置成功
```

4.7.6 控制手指各关节速度

- 说明: 手指各个关节的位置力矩, 以百分比的方式。

- 控制方式: service

- 服务名称: `/inspire_hand/set_speed/left_hand`
和 `/inspire_hand/set_speed/right_hand`

- 数据定义位置: `bodyctrl_msgs::srv::set_speed.srv`

- request:

代码块

```
1 float32 speed0_Ratio #小拇指-百分比 (1代表800ms从最大角度到最小角度, 0.5代表1600ms, 0.25代表3200ms)
2 float32 speed1_Ratio #无名指-百分比
3 float32 speed2_Ratio #中指-百分比
4 float32 speed3_Ratio #食指-百分比
5 float32 speed4_Ratio #大拇指弯曲-百分比
6 float32 speed5_Ratio #大拇指旋转-百分比
```

- response:

代码块

```
1 bool speed_accepted #是否设置成功
```

4.7.7 清除手指各关节错误

- 说明：清除手指各个关节的错误接口。
- 控制方式：service
- 服务名称： `/inspire_hand/set_clear_error/left_hand` 和 `/inspire_hand/set_clear_error/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_clear_error.srv`
- request: `null`
- response:

代码块

```
1 bool setclear_error_accepted #是否设置成功
```

4.8 六维力

4.8.1 状态获取接口

1. 六维力信息

- 说明：获取六维力当前的信息。其中上报频率为1000Hz。
- 控制方式：topic
- 话题名称： `/arm_6dof_left` 和 `/arm_6dof_right`
- 类型： `geometry_msgs::msg::WrenchStamped.msg`
- 数据格式：

代码块

```
1 Header header uint32 seq #序列号
2 Header header time stamp #时间戳
3 Header header string frame_id #帧id
4
5 Wrench wrench Vector3 force #3个方向的力
6 Wrench wrench Vector3 torque #3个方向的力矩
```

4.9 lyre 语音包

4.9.1 程序包及安装位置

lyre 语音包包括 lyre_ros2、lyre_msgs 和 lyre_examples 三个程序包。

- lyre_ros2 是语音包的功能程序包，安装在 orin 主控中。
- lyre_msgs 是语音包的接口定义。该接口定义同时安装在 x86 的 ros2ws 目录以及 lyre_ros2 目录。
- lyre_examples 是语音包接口的调用示例程序。该示例程序安装在 x86 的 ros2ws 目录。

lyre 语音包程序的目录结构如下：

代码块

```
1  |— ros2ws/                # x86
2  |   |— install
3  |   |   |— lyre_msgs      # 接口定义
4  |   |   |   |— README.md # SDK 开发文档
5  |   |   |   |— lyre_examples # 接口调用示例程序
6  |
7  |— lyre_ros2/            # orin
8  |   |— install
9  |   |   |— lyre_msgs      # 接口定义
10 |   |   |   |— README.md # SDK 开发文档
11 |   |   |   |— lyre       # 功能包
12 |   |— scripts
13 |   |   |— setup_network.sh # 网络配置脚本
14 |   |   |— setup_systemd.sh # 自启动服务部署脚本
15 |   |   |— reset_systemd.sh # 自启动服务卸载脚本
16 |   |— README.md          # 用户手册
17 |   |— QAs.md              # 常见问题
```

4.9.2 语音交互功能

机器人提供有语音交互功能。设备开机以后，语音交互功能为关闭状态。通过遥控器按键可以开启语音交互功能。

4.9.2.1 遥控器开关

E、G、H 键在中间；F 键向上。

长按 A 键，可以打开/关闭语音交互功能。

4.9.2.2 语音对话方式

站立在机器人正前方 1.5 米左右的位置，正视机器人胸前麦克风上方的摄像头，与机器人进行对话。

4.9.3 启动语音包

语音包的 ROS 包名为 lyre，在 systemd 中的服务名称也是 lyre。

4.9.3.1 手动启动语音对话的 ROS 节点

在 `lyre_ros2` 程序包中使用下面的命令启动语音对话的 ROS 节点。

代码块

```
1 cd lyre_ros2
2 source install/setup.bash
3 ros2 launch lyre chat.launch.py
```

注意：机器人默认会在开始时自行启动 `lyre` 节点。同一时间只允许一个 `lyre` 进程运行。通过下面的命令检查系统中是否有其它 `lyre` 进程正在运行。

代码块

```
1 ps -ef | grep "ros2 launch lyre" | grep -v grep
```

如果有打印内容，则说明 `lyre` 进程正在运行。通过下面的命令可以关闭系统中正在运行的 `lyre` 进程。

代码块

```
1 pkill -INT -f "ros2 launch lyre"
```

注意：如果通过 `systemd` 配置了开机自启动服务，则需要在关闭自启动服务以后再结束进程。否则，手动结束的进程会被重新启动。具体操作参考后面的章节。

4.9.3.2 开机自启动服务

出厂时，语音包的 `systemd` 服务 `lyre` 会在开机时自行启动。

检查 `lyre` 服务是否会在开机时自启动

代码块

```
1 systemctl is-enabled lyre
```

如果显示 `enabled`，则说明 `lyre` 服务会在开机时自启动。

打开 `lyre` 服务开机自启动设置

代码块

```
1 sudo systemctl enable lyre
```

关闭 lyre 服务开机自启动设置

代码块

```
1 sudo systemctl disable lyre
```

4.9.4 启动模式

`lyre` 语音包提供了四种运行模式，分别是 `play`、`asr`、`audio`、`chat`。接入者可以根据实际情况选用其中的一种。

注意：启动模式是互斥的。同一时间，只可以允许 4 种启动模式中的一种。

- `play`: 在此启动模式下，仅『音频播放』相关的功能可以使用。
- `asr`: 在此启动模式下，仅『语音识别』相关的功能可以使用。
- `audio`: 在此启动模式下，可以同时使用『语音识别』和『音频播放』功能，包括『文本播放』功能。
- `chat`: 在此启动模式下，可以同时使用『语音识别』、『音频播放』和『语音对话』功能。

不同模式的节点的启动命令统一为：

代码块

```
1 ros2 launch lyre <mode>.launch.py
```

例如，启动 `audio` 模式的命令为：

代码块

```
1 ros2 launch lyre audio.launch.py
```

部署某个启动模式的开机自启动服务

`lyre_ros2` 程序包给出了自启动服务的部署脚本 `setup_systemd.sh`，该脚本位于 `lyre_ros2/scripts` 目录下。使用方法如下：

代码块

```
1 bash setup_systemd.sh <mode>[default:chat]
```

例如：将 `play` 启动模式部署为自启动服务的命令如下：

```
1 bash setup_systemd.sh play
```

代码块

4.9.5 接口示例程序

`lyre_examples` 程序包提供了部分 `CPP` 版本的接口调用示例。程序包的目录结构如下：

代码块

```
1  lyre_examples/
2  |— bin/                # 接口调用示例的可执行程序
3  |   |— lyre_example_play_file_srv
4  |— cli/                # 接口调用示例的终端执行脚本
5  |   |— call_play_file_srv.sh
6  |   |— sub_play_progress_msg.sh
7  |— src/                # 接口调用示例的 CPP 代码
8  |   |— play_file_srv.cpp
9  |— res/                # 静态资料
10 |   |— test.mp3
11 |— CMakeLists.txt      # 项目 CMakeLists.txt 示例
12 |— package.xml         # 项目 ROS package.xml 示例
```

对于每个开放的语音接口，`lyre_examples` 包都提供有接口调用的示例代码、编译后的可执行程序、终端调用脚本，分别放置在 `src`、`bin`、`cli` 目录。

为了避免混淆，每个演示所用的可执行程序的文件名前面都加了 `lyre_example_` 作为前缀。每个用于订阅 Topic 的脚本的文件名都加了 `sub_` 作为前缀；每个用于发布 Topic 的脚本的文件名都加了 `pub_` 作为前缀。每个用于调用 Service 的脚本的文件名则都加了 `call_` 作为前缀。

对于二次开发的 CPP ROS2 项目，可以参考 `lyre_examples` 包中的方式导入 `lyre_msgs` 接口，再参考示例代码完成功能开发。

4.9.5.1 ROS package 配置

代码块

```
1 <package format="3">
2   <depend>lyre_msgs</depend>
3 </package>
```

4.9.5.2 项目 CMake 配置

代码块

```
1 find_package(lyre_msgs REQUIRED)
2
```

```

3   ament_target_dependencies(
4     <your_target>
5     lyre_msgs
6   )

```

4.9.5.3 编译时、运行时依赖配置

x86 的 `ros2ws` 和 orin 的 `lyre_ros2` 提供了 `lyre_msgs` 包。在构建或者运行自己的 ROS 程序之前，需要先从这两个工作空间导入 `lyre_msgs` 依赖。

代码块

```

1   # x86
2   source ros2ws/install/setup.bash
3
4   # orin
5   source lyre_ros2/install/setup.bash

```

4.9.6 音频播放接口

4.9.6.1 播放本地文件（Service）

如果你已经明确知道 `lyre_ros2` 程序包所在的主控板中有一个音频文件，你可以使用下面的 ROS 服务播放这个文件。

代码块

```

1   # lyre_msgs/srv/PlayFile.srv
2   # 以本地文件的形式播放一段语音。
3
4   string SERVICE_NAME = /audio_play/play_file
5
6   string sid      # Stream identifier (unique per audio stream).
7   uint32 seq      # Sequence number (incremental per packet).
8   bool  last      # Last flag (true if this is the final packet).
9   bool  force     # Force playback (stop all running tasks and play immediately).
10
11  string path      # Absolute path to the audio file in the local file system.
12  ---
13
14  int8 CODE_OK = 0
15  int8 CODE_INVALID_PARAMS = 1
16  int8 CODE_FAILED = -1
17
18  string sid        # Playback stream ID (generated internally if absent).
19  int8  code        # Status code.

```

```
20 string message # Human-readable status message.
```

接口调用的 C++ 示例代码请参考 `lyre_examples` 程序包中的 `play_text_srv`。

4.9.6.2 播放远程文件 (Service)

你还可以使用下面的 ROS 服务来播放一个来自网络的音频文件。

代码块

```
1 # lyre_msgs/srv/PlayUrl.srv
2 # 以文件 URL 的形式播放一段语音。
3
4 string SERVICE_NAME = /audio_play/play_url
5
6 string sid      # Stream identifier (unique per audio stream).
7 uint32 seq      # Sequence number (incremental per packet).
8 bool  last      # Last flag (true if this is the final packet).
9 bool  force      # Force playback (stop all running tasks and play immediately).
10
11 string url      # URL to the audio file in the network.
12 ---
13
14 int8 CODE_OK = 0
15 int8 CODE_INVALID_PARAMS = 1
16 int8 CODE_FAILED = -1
17
18 string sid      # Playback stream ID (generated internally if absent).
19 int8  code      # Status code.
20 string message  # Human-readable status message.
```

接口调用的 C++ 示例代码请参考 `lyre_examples` 程序包中的 `play_url_srv`。

4.9.6.3 播放文本 (Service)

如果你需要从别的主控板访问音频播放服务，可以使用下面的 ROS 服务来播放一个字符串。

代码块

```
1 # lyre_msgs/srv/PlayText.srv
2 # 以文本的形式播放一段语音。
3
4 string SERVICE_NAME = /audio_play/play_text
5
6 string sid      # Stream identifier (unique per audio stream).
7 uint32 seq      # Sequence number (incremental per packet).
8 bool  last      # Last flag (true if this is the final packet).
```

```

9  bool    force    # Force playback (stop all running tasks and play immediately).
10
11  string  text     # Text to be synthesized into speech.
12
13  string  token    # System api (unavailable for applications).
14  string  output   # System api (unavailable for applications).
15  ---
16
17  int8    CODE_OK   = 0
18  int8    CODE_INVALID_PARAMS = 1
19  int8    CODE_FAILED = -1
20
21  string  sid       # Playback stream ID (generated internally if absent).
22  int8    code      # Status code.
23  string  message   # Human-readable status message.

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_text_srv`。

注意：此功能仅在 `audio` 和 `chat` 启动模式下才可用。

4.9.6.4 监听播放事件 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放事件：

代码块

```

1  # lyre_msgs/msg/PlayEvent.msg
2  # Play 音频播放的事件。
3
4  string TOPIC_NAME = /audio_play/event
5
6  int8    EVENT_STARTED    = 0
7  int8    EVENT_COMPLETED  = 1
8  int8    EVENT_STOPPED    = 2
9  int8    EVENT_CANCELLED  = 3
10 int8    EVENT_FAILED     = 4
11
12 string  sid
13 uint32  seq
14 int8    event
15 string  message

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_event_msg`。

4.9.6.5 监听播放进度 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放的进度及总时长（单位：秒）：

代码块

```
1 # lyre_msgs/msg/PlayProgress.msg
2 # Play 音频播放的进度。
3
4 string TOPIC_NAME = /audio_play/progress
5
6 string sid
7 uint32 seq
8 float64 position
9 float64 duration
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_progress_msg`。

4.9.6.6 停止音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（停止以后无法恢复）：

代码块

```
1 # lyre_msgs/srv/PlayStop.srv
2 # 停止当前的音频播放任务（不可恢复）。
3
4 string SERVICE_NAME = /audio_play/stop
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_stop_srv`。

4.9.6.7 暂停音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（暂停以后可以调用 Resume 服务恢复播放）：

代码块

```
1 # lyre_msgs/srv/PlayPause.srv
2 # 暂停当前的音频播放任务。
3
4 string SERVICE_NAME = /audio_play/pause
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_pause_srv`。

4.9.6.8 恢复音频播放 (Service)

通过下面的 ROS 服务来恢复播放暂停中的音频：

代码块

```
1 # lyre_msgs/srv/PlayResume.srv
2 # 恢复已暂停的音频播放任务。
3
4 string SERVICE_NAME = /audio_play/resume
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_resume_srv`。

4.9.7 语音识别接口

设备集成有语音识别模块，可以提供 ASR 语音识别功能。

4.9.7.1 监听唤醒事件 (Topic)

在设备周围喊出关键词，可以触发唤醒事件。不同设备的唤醒关键词请参考用户手册。

在代码中可以通过以下 ROS 话题来监听唤醒消息：

代码块

```
1 # lyre_msgs/msg/AsrKeyword.msg
2 # ASR 唤醒时识别到的关键字。
3
4 string TOPIC_NAME = /audio_asr/keyword
5
6 string keyword
7 int32 angle
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_keyword_msg`。

4.9.7.2 监听语音识别结果 (Topic)

设备可以将周围的声音转化为文本。可以通过以下 ROS 话题获取识别到的文本：

代码块

```
1 # lyre_msgs/msg/AsrIat.msg
2 # ASR 语音识别到的内容。
3
4 string TOPIC_NAME = /audio_asr/iat
5
6 string id
7 string text
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_iat_msg`。

4.9.7.3 监听其它事件 (Topic)

设备还会发布其它 ASR 相关的事件，事件类型定义在下面的 ROS 话题中：

代码块

```
1  # lyre_msgs/msg/AsrEvent.msg
2  # ASR 设备报出的事件。
3
4  string TOPIC_NAME = /audio_asr/event
5
6  int8 EVENT_ERROR    = 2 # 出错事件。arg1 是错误码。
7  int8 EVENT_STATE    = 3 # 服务状态事件。
8  int8 EVENT_WAKEUP   = 4 # 唤醒事件。
9  int8 EVENT_SLEEP    = 5 # 休眠事件。
10
11  int8 EVENT_PRE_SLEEP      = 10 # 准备休眠事件。
12  int8 EVENT_CONNECTED_TO_SERVER = 13 # 与服务端建立连接。
13  int8 EVENT_SERVER_DISCONNECTED = 14 # 与服务端断开连接。
14
15  int8 event
16  int8 arg1
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_event_msg`。

4.9.7.4 aiui_msg 事件

SDK 兼容旧版本的 `/xunfei/aiui_msg` 话题，这个话题已经被标记为 Deprecated，未来很可能会被移除。

代码块

```
1  string TOPIC_NAME = /xunfei/aiui_msg
```

该话题的类型是 `std_msgs::msg::String`，可以通过下面的示例脚本获取到：

代码块

```
1  ros2 topic echo -f /xunfei/aiui_msg
```

4.9.8 语音对话接口

4.9.8.1 语音交互开关

`lyre` 节点接收一个 ROS 话题来控制语音交互的可用状态。

代码块

```
1 string TOPIC_NAME = /audio_chat/enable
```

话题的类型是 `std_msgs/msg/Bool`。

对话开启的示例脚本：

代码块

```
1 ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: true'
```

对话关闭的示例脚本：

代码块

```
1 ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: false'
```

4.10 深度相机

启动奥比相机方法-ROS2

Orin (192.168.41.2) :

代码块

```
1 cd /home/nvidia/orbbec_camera_ros2
2 source install/setup.bash
3 ros2 launch orbbec_camera gemini_330_series.launch.py
```

4.11 电池状态及电量获取

1. 电池状态

- 说明：获取电池状态信息。其中上报频率为1Hz。
- 充电状态：`master_battery_current`字段，负值表示在放电，正值表示在充电
- 控制方式：topic
- 话题名称：`/power/battery/status`

- 类型: `bodyctrl_msgs::msg::PowerBatteryStatus`

- 数据格式:

代码块

```
1  # @brief 电源板电压、电流、电量、温度等信息
2
3  std_msgs/Header header
4
5  # battery status information
6  int32 battery_installed # 0x00都不存在, 0x01小电池存在, 0x02大电池存在, 0x03大小电
   池都存在
7  int32 battery_working   # 0x10大电池, 0x01小电池
8
9  float32 master_battery_voltage # 电池电压信息
10 float32 master_battery_current # 电池电流信息, 负值表示在放电, 正值表示在充电
11 float32 master_battery_power   # 电池电量
12
13 float32 little_battery_voltage # 小电池电压信息
14 float32 little_battery_current # 小电池电流信息, 负值表示在放电, 正值表示在充电
15 float32 little_battery_power   # 小电池电量
16
17 int8 pg12a # 00低, 01高
18 int8 pg12b # 00低, 01高
19 int8 pg12c # 00低, 01高
20 int8 pg12d # 00低, 01高
21 int8 pg5cd # 00低, 01高
22 int8 pg5ab # 00低, 01高
23
24 int8 pgrdc2   # 00低, 01高
25 int8 pgrdc1   # 00低, 01高
26 int8 pgheader # 00低, 01高
27 int8 pgbutton2 # 00低, 01高
```

2. 电源板状态

- 说明: 获取电源板状态信息。其中上报频率为1Hz。

- 控制方式: topic

- 话题名称: `/power/board/status`

- 类型: `bodyctrl_msgs::msg::PowerStatus`

- 数据:

代码块

```
1  # @brief 电源板电压、电流、电量、温度等信息
2
3  std_msgs/Header header
4
5  # temperature information
6  float32 waist_temp # 腰部MOS温度
7  float32 arm_a_temp # 臂A MOS温度
8  float32 arm_b_temp # 臂B MOS温度
9  float32 leg_a_temp # 腿A MOS温度
10 float32 leg_b_temp # 腿B MOS温度
11
12 float32 waist_temp_max # 腰部MOS温度最大值
13 float32 arm_a_temp_max # 臂A MOS温度最大值
14 float32 arm_b_temp_max # 臂B MOS温度最大值
15 float32 leg_a_temp_max # 腿A MOS温度最大值
16 float32 leg_b_temp_max # 腿B MOS温度最大值
17
18 float32 waist_temp_min # 腰部MOS温度最小值
19 float32 arm_a_temp_min # 臂A MOS温度最小值
20 float32 arm_b_temp_min # 臂B MOS温度最小值
21 float32 leg_a_temp_min # 腿A MOS温度最小值
22 float32 leg_b_temp_min # 腿B MOS温度最小值
23
24 # current information
25 float32 arm_a_curr # 臂A电流
26 float32 arm_b_curr # 臂B电流
27 float32 leg_a_curr # 腿A电流
28 float32 leg_b_curr # 腿B电流
29 float32 waist_curr # 腰部电流
30 float32 head_curr # 头部电流
31
32 float32 arm_a_curr_max # 臂A电流最大值
33 float32 arm_b_curr_max # 臂B电流最大值
34 float32 leg_a_curr_max # 腿A电流最大值
35 float32 leg_b_curr_max # 腿B电流最大值
36 float32 waist_curr_max # 腰部电流最大值
37 float32 head_curr_max # 头部电流最大值
38
39 float32 arm_a_curr_min # 臂A电流最小值
40 float32 arm_b_curr_min # 臂B电流最小值
41 float32 leg_a_curr_min # 腿A电流最小值
42 float32 leg_b_curr_min # 腿B电流最小值
43 float32 waist_curr_min # 腰部电流最小值
44 float32 head_curr_min # 头部电流最小值
45
46 float32 arm_a_volt # 臂A电压
47 float32 arm_b_volt # 臂B电压
```

```

48 float32 leg_a_volt #腿A电压
49 float32 leg_b_volt #腿B电压
50 float32 waist_volt #腰部电压
51 float32 bus_volt #母线电压
52
53 float32 arm_a_volt_max #臂A电压最大值
54 float32 arm_b_volt_max #臂B电压最大值
55 float32 leg_a_volt_max #腿A电压最大值
56 float32 leg_b_volt_max #腿B电压最大值
57 float32 waist_volt_max #腰部电压最大值
58 float32 bus_volt_max #母线电压最大值
59
60 float32 arm_a_volt_min #臂A电压最小值
61 float32 arm_b_volt_min #臂B电压最小值
62 float32 leg_a_volt_min #腿A电压最小值
63 float32 leg_b_volt_min #腿B电压最小值
64 float32 waist_volt_min #腰部电压最小值
65 float32 bus_volt_min #母线电压最小值
66
67 # version information
68 string software_version # 软件版本
69 string hardware_version # 硬件版本
70
71 # battery status information
72 float32 battery_voltage # 电池电压信息
73 float32 battery_current # 电池电流信息
74 float32 battery_power # 电池电量
75

```

3. 急停按键状态

- 说明：获取急停按键状态信息。其中上报频率为1Hz。
- 控制方式：topic
- 话题名称：/power/board/key_status
- 类型：`bodyctrl_msgs::msg::PowerBoardKeyStatus.msg`
- 数据格式：

代码块

```

1 # @brief 电源板按键状态
2
3 std_msgs/Header header
4
5 # power board status
6 uint32 work_time # 工作时间

```

```

7 std_msgs/Bool is_estop          # 急停是否被按下
8 std_msgs/Bool is_remote_estop  # 软急停是否被按下
9 std_msgs/Bool is_power_on      # 电源是否正常供电

```

4.12 遥控器接口

1. 遥控器事件数据

- 说明：获取遥控器原始数据
- 控制方式：topic
- 话题名称：/sbus_data/event
- 类型： `bodyctrl_msgs::msg::SbusData.msg`
- 数据格式：

代码块

```

1  int32 KEY_NONE=0
2  int32 KEY_A_UP=1      # A键抬起事件
3  int32 KEY_A_DOWN=2    # A键按下事件
4  int32 KEY_B_UP=3      # B键抬起事件
5  int32 KEY_B_DOWN=4    # B键按下事件
6  int32 KEY_C_UP=5      # C键抬起事件
7  int32 KEY_C_DOWN=6    # C键按下事件
8  int32 KEY_D_UP=7      # D键抬起事件
9  int32 KEY_D_DOWN=8    # D键按下事件
10 int32 KEY_E_UP=9      # E键上拨事件
11 int32 KEY_E_MID=10     # E键回中事件
12 int32 KEY_E_DOWN=11   # E键下拨事件
13 int32 KEY_F_UP=12     # F键上拨事件
14 int32 KEY_F_MID=13     # F键回中事件
15 int32 KEY_F_DOWN=14   # F键下拨事件
16 int32 KEY_G_LEFT=15   # G键左拨事件
17 int32 KEY_G_MID=16     # G键回中事件
18 int32 KEY_G_RIGHT=17  # G键右拨事件
19 int32 KEY_H_LEFT=18   # H键左拨事件
20 int32 KEY_H_MID=19     # H键回中事件
21 int32 KEY_H_RIGHT=20  # H键右拨事件
22
23 std_msgs/Header header
24
25 int32 key_event_new    # 按键事件新值
26 int32 key_event_old    # 按键事件旧值
27                        # 例如 G键从右边拨到中间位置，则key_event_old=KEY_G_RIGHT，
28                        key_event_new=KEY_G_MID

```

```

29  int8 button_a      # A键键值, -1: 松开, 1: 按下
30  int8 button_b      # B键键值, -1: 松开, 1: 按下
31  int8 button_c      # C键键值, -1: 松开, 1: 按下
32  int8 button_d      # D键键值, -1: 松开, 1: 按下
33  int8 button_e      # E键键值, <-0.5: 上拨, -0.5~0.5: 中间, >0.5: 下拨
34  int8 button_f      # F键键值, <-0.5: 上拨, -0.5~0.5: 中间, >0.5: 下拨
35  int8 button_g      # G键键值, <-0.5: 左拨, -0.5~0.5: 中间, >0.5: 右拨
36  int8 button_h      # H键键值, <-0.5: 右拨, -0.5~0.5: 中间, >0.5: 左拨
37
38  float32 x1          # 左摇杆X方向值 (左右方向), 范围-1.0~1.0
39  float32 y1          # 左摇杆Y方向值 (上下方向), 范围-1.0~1.0
40  float32 x2          # 右摇杆X方向值 (左右方向), 范围-1.0~1.0
41  float32 y2          # 右摇杆Y方向值 (上下方向), 范围-1.0~1.0

```

4.13 软件版本信息获取

完整版本信息文件（包含x86、orin、firmware等完整版本信息描述文件）定义在本地软件包的安装目录中：

ros2路径是：/home/ubuntu/ros2ws/version_info.json

格式如下：

代码块

```

1  {
2    "product": "TG2.0-XXX",
3    "version": "vX.X.X",
4    "commit": "bu/integration/tiangong2.0/XXX,xxx",
5    "release_time": "xxxx-xx-xx xx:xx:xx",
6    "x86": [
7      {
8        "module": "benti",
9        "status": "enable",
10       "product": "TG2.0-XXX",
11       "platform": "linux-x86",
12       "version": "vX.X.X",
13       "file_name": "TG2.0-Pro_linux-x86_general_vX.X.X.tar.gz",
14       "commit": "bu/project/tiangong2.0/pro",
15       "install_dir_name": "ros2ws",
16       "install_address": "192.168.41.1",
17       "release_time": "XXXX-XX-XX XX:XX:XX"
18     },
19     {
20       "module": "ota",
21       "status": "disable",
22       "product": "ota",

```

```

23     "platform": "linux-x86",
24     "version": "vX.X.X",
25     "file_name": "ota_linux-x86_general_vX.X.X.tar.gz",
26     "commit": "bu/base/ota",
27     "install_dir_name": "ota",
28     "install_address": "192.168.41.1",
29     "release_time": "XXXX-XX-XX XX:XX:XX"
30 }
31 ]
32 ...
33 }

```

4.14 bag录制接口

直接使用launch文件可以启动ros2 topic录制，并存储到默认路径（/home/ubuntu/bags）。录制按照100M bag分片，录制目录最大存储容量为4G，超过限制后会自动删除最早文件夹以及bag文件。

代码块

```

1  #运行指定topic录制，录制列表存储在
   /home/ubuntu/ros2ws/install/utils/lib/utils/bag_record/config/record.json
   /topic_list，默认列表中存储了本体控制的关键topic
2  ros2 launch utils record_trigger.py

```

4.15 导航接口

<https://wiki.slamtec.com/display/SD/Slamware+Solution>

<https://wiki.slamtec.com/display/SD/KBSW180113+RoboStudio+User+Manual>

4.16 底盘接口

底盘目前提供两种控制方式：

1. rostopic控制方式：https://developer.slamtec.com/docs/slamware/ros-sdk/2.8.2_rtm/slamware_ros_sdk_server_node/

📖 [基于 WSL Ubuntu22.04 使用 ROS2 SDK控制思岚科技底盘的详细指南](#)

2. http控制方式：<https://docs.slamtec.com>

主机设置192.168.11/24网段，浏览器访问192.168.11.1:1448